

Algoritmos y Estructuras de Datos I

4/10/2022

Repaso: Modelo computacional de la programación imperativa

En programación imperativa un programa es una “receta” o una “serie de pasos” y la ejecución de un programa parte de un **estado** inicial y lo va transformando hasta llegar a un **estado** final.

Estado: es una asignación de valores a un conjunto fijo y predeterminado de variables. En un programa, las variables que componen el estado (con su nombre y tipo) se definen al principio en lo que se llama la “declaración de constantes y variables”.

Programa de ejemplo:

```
Var x, y : Int           ← declaración de variables
ℓ1  x := 3 ;             ← sentencias del programa (la “receta”)
ℓ2  y := x + 3
```

Variables que componen el estado: “x” e “y” de tipo Int.

Ejemplo de estado inicial: $\sigma_0: x \mapsto 4, y \mapsto 8$

(observación: **siempre** ponemos valores en los estados, nunca expresiones)
(el estado me indica qué contiene la memoria de la computadora en un momento determinado)

Ejemplo de ejecución del programa a partir de este estado inicial:

línea	estado	observaciones
	$\sigma_0: x \mapsto 4, y \mapsto 8$	estado inicial
ℓ ₁ x := 3	$\sigma_1: x \mapsto 3, y \mapsto 8$	
ℓ ₂ y := x + 3	$\sigma_2: x \mapsto 3, y \mapsto 6$	estado final

Siempre el estado final de la ejecución de un programa depende tanto del programa como del estado inicial.

Un programa imperativo es un “**transformador de estados**”.

Repaso: Sintaxis vs. Semántica

Siempre que hablemos de lenguajes de programación tendremos estos dos aspectos:

Sintaxis: ¿Cómo se escriben los programas? Un programa es un texto (una secuencia de letras). La sintaxis de un lenguaje me dice qué textos son programas válidos.

Semántica: ¿Qué significan los programas? ¿Qué hacen? Un programa se puede ejecutar, y esa ejecución tiene un efecto sobre un “mundo semántico”. En el caso de la programación imperativa, el mundo semántico es el estado.

Repaso: Ejemplo: contar múltiplos de 6

Vamos a hacer un programa que usa todo lo que acabamos de ver.

Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.

Idea del algoritmo: Con un ciclo, recorrer los números desde 0 hasta N (o al revés, desde N hasta 0). Para cada valor, fijarme si es múltiplo de 6 o no. Si sí lo es, sumamos 1 al resultado.

Programa:

```
Const N : Int ;
Var numero_actual, resultado : Int ;
numero_actual , resultado := 0 , 0 ;
do numero_actual ≤ N →
    if numero_actual mod 6 = 0 →
        resultado := resultado + 1 ;
    [] numero_actual mod 6 ≠ 0 →
        skip
    fi ;
    numero_actual := numero_actual + 1
od
```

Importante: Indentación!! Organizar y tabular el código para que pueda leerse correctamente.

Ejercicio: verificar haciendo la ejecución con un ejemplo concreto de estado inicial (valores para **N**, numero_actual y resultado).

Arreglos

Es el único tipo de dato que me permite tener una colección de valores de algún tipo (en el lenguaje imperativo del teórico/práctico). Como de costumbre, la semántica de la programación imperativa se acerca más a la verdadera arquitectura de las computadoras, y

los arreglos son el tipo de colecciones más cercano al verdadero funcionamiento de la memoria de una computadora.

El **arreglo** define una colección de valores **ordenada** y de **tamaño fijo**.

Es como una lista de programación funcional pero de **largo fijo**. No existe el concepto de agregar o quitar elementos en un arreglo (tampoco de concatenar arreglos).

Es como una tupla pero todas las componentes son del mismo tipo.

En **Algoritmos I** usaremos los arreglos sólo para leer su información y no para modificarlos, escribirlos o calcular resultados contenidos en ellos.

Declaración (tipo)

Sintaxis: `Array[N, M) of <tipoX>`

Semántica: Tengo un arreglo de (M - N) elementos de tipo <tipoX>, cuyos índices son N, N+1, N+2, ..., M-1.

Observaciones:

- <tipoX> puede ser cualquier tipo válido, incluso otro arreglo (e.g. para definir matrices o arreglos multidimensionales).
- los arreglos son siempre “cerrado/abierto” (o sea “[/ “”).
- por lo general usaremos letras mayúsculas para variables que contienen arreglos.
- por lo general usaremos N = 0.

Ejemplos:

```
Const A : Array[0 , 4) of Int ;  
    // este arreglo tiene elementos A.0, A.1, A.2 y A.3  
  
Var MiArreglo : Array[11, 15) of Bool;  
  
Const N, M : Nat ;  
Var Matriz : Array[0, N) of Array[0, M) of Int;  
    // tenemos una matriz de N x M números enteros.
```

Consulta / acceso (expresión)

Si yo tengo un arreglo, me interesa acceder a los valores para usarlos en una expresión (por ejemplo para usar la expresión en una asignación o en una guarda de un if o un do).

Sintaxis: Si mi arreglo se llama A , y tengo una expresión E que es tipo entero, la sintaxis es:

A.E

Semántica:

1. Calcular el valor entero asociado a la expresión E. Sea n este valor.
2. "A.E" me devuelve el valor alojado en la n-ésima posición del arreglo A.
(este valor devuelto es del tipo <tipoX>)

Observaciones:

- si E resulta en un valor n que no es una posición válida del arreglo, el acceso da error (el programa se rompe).

Ejemplo:

Si tengo:

```
Const A : Array[0 , 4) of Int ;
Var x : Int ;
```

Puedo hacer:

```
x := x + A.3 * 2
```

Otro ejemplo:

Si tengo una matriz de 3x3:

```
Var Matriz : Array[0, 3) of Array[0, 3) of Int;
```

```
Var x : Int ;
```

Puedo hacer la suma de la diagonal así:

```
x := (A.0).0 + (A.1).1 + (A.2).2
```

Asignación para arreglos (sentencia)

Con la sentencia de asignación para arreglos puedo modificar un valor en una posición.

Sintaxis:

$$A.E := F$$

donde E es una expresión de tipo Nat, y F es una expresión de tipo <tipoX> a donde <tipoX> es el tipo de los elementos del arreglo.

Semántica:

1. Calcular los valores asociados a E y a F.
2. Modificar el estado, cambiando el valor asociado a la posición E por el resultado de F.

Observaciones:

- esta asignación es una asignación distinta a la normal y no se pueden mezclar.
- **NO LO VAMOS A USAR EN ALGORITMOS I.**

Ejemplo: Sumar elementos de un arreglo

Supongamos que tengo $N > 0$ y un arreglo de N elementos de tipo entero. Quiero calcular la suma de todos los elementos.

Idea del programa: Con un ciclo, usamos una variable para recorrer las posiciones del arreglo, desde la primera hasta la última, y en otra variable vamos calculando la suma.

```
Const N : Int, A : Array[0, N) of Int;  
Var pos, res : Int;  
res, pos := 0, 0 ;  
do pos < N →  
    res := res + A.pos ;  
    pos := pos + 1  
od
```

Equivalentemente:

```
Const N : Int, A : Array[0, N) of Int;  
Var pos, res : Int;  
res, pos := 0, 0 ;  
do pos < N →  
    res, pos := res + A.pos, pos + 1  
od
```

Uno que no anda:

```
Const N : Int, A : Array[0, N) of Int;  
Var pos, res : Int;  
res, pos := 0, 0 ;  
do pos < N →  
    pos := pos + 1  
    res := res + A.pos ;  
od
```

Este no anda porque se saltea el primer elemento y además da error porque intenta acceder a la posición "N" que no es una posición válida.

Ejercicio: Verificar los tres programas con un ejemplo de un arreglo en concreto.

Programación imperativa: Lenguaje completo

Sintaxis de un programa. Dos secciones principales:

1. Declaración de constantes y variables
2. Sentencia del programa

Semántica de un programa:

- Estado: variables y constantes definidas por las declaraciones.

- Ejecución: Dado un estado inicial, ejecutar la sentencia del programa, modificando el estado hasta llegar a un estado final.

Diferencias con funcional: En imperativo (del teórico) no hay funciones y **no hay un valor resultado**. El **resultado es el estado final entero**.

Diferencias con C: En imperativo (del teórico) no hay funciones ni procedimientos. Sólo estamos tomando de C lo que se llama programación “in the small” (en pequeña escala): Los algoritmos pequeños, sin la superestructura de los programas que podemos definir en C. Tampoco vamos a hacer input/output.

Declaración de constantes y variables

Especificador: `Var` o `Const`. (para avisar si declaro variable o constante)

Nombres (identificadores): Palabras con letras y números (pero empezando con una letra).

Tipos: `Nat`, `Int`, `Real`, `Bool`, `Char`, arreglos. **No hay listas**.

Sintaxis:

```
Var/Const  nombre1, nombre2, ... , nombren  :  Tipo;
...
Var/Const  nombre1, nombre2, ... , nombren  :  Tipo;
```

Ejemplo:

```
Var x, y : Int;
Var resultado : Bool;
```

Otro ejemplo:

```
Const divisor, dividendo : Int;
Var cociente, resto : Int;
```

Observación: El valor de las constantes no estará determinado en los programas (no lo escribiremos como parte de la sintaxis), si no que podrá ser cualquier valor (en principio). Luego veremos cómo se podrá restringir los valores posibles. (por ejemplo: me interesa que el divisor no sea cero).

Otro ejemplo (con variables de varios tipos en la misma línea):

```
Var x : Int, b : Bool;
```

Sentencias

Dos tipos:

- Elementales: sentencias básicas que no necesitan de otras sentencias.

- Compuestas: sentencias que se arman a partir de otras sentencias (o sea, sentencias que tienen adentro otras sentencias).

Skip

Es una sentencia elemental que no hace nada.

Sintaxis:

skip

Semántica: No modifica el estado.

Asignación (:=)

Sentencia elemental que se usa para asignar valores a variables.

Sintaxis:

v1, v2, ..., vn := E1, E2, ..., En

a donde v1, v2, ..., vn son variables declaradas del programa, y E1, E2, ..., En son expresiones válidas en lenguaje y del tipo que se corresponde con las variables.

Semántica:

1. Se calculan valores-resultado para todas las expresiones E1, E2, ..., En. (O sea, se hace usando el estado actual, no el nuevo estado.)
2. Se modifica el estado, asignando a cada variable vi el resultado de calcular la expresión Ei.

Expresiones válidas en programación imperativa:

- valores constantes
- variables y constantes declaradas
- operaciones básicas (+, -, *, /, div, mod, max, min, $\wedge$, $\vee$, $\neg$, =, $\neq$, $\leq$, $<$, $>$, etc.)
- accesos a elementos de arreglos
- **NO: llamadas a funciones, cuantificadores ni cosas que esconden cuantificadores (factorial, exponenciación, etc.)**

Secuenciación (;)

Primera sentencia compuesta que vemos. La secuenciación me permite ejecutar una sentencia y después otra.

Sintaxis:

S1 ; S2

donde S1 y S2 son dos sentencias.

Semántica:

1. Ejecutar S1 a partir del estado inicial.
2. Ejecutar S2 a partir del estado que resulta del paso 1. (el estado queda como lo deja la ejecución de S2).

Observaciones:

- La secuenciación es **asociativa**:

Es lo mismo (semánticamente)

$(S1 ; S2) ; S3$

que

$S1 ; (S2 ; S3)$

Luego, podremos escribir directamente:

$S1 ; S2 ; S3$

- Un programa es en realidad siempre **una sola sentencia**. Cuando hablamos de varias sentencias en realidad éstas forman una sola sentencia a través de la secuenciación.

Condicional (if)

Sentencia compuesta. El condicional me permite ejecutar diferentes sentencias dependiendo de una condición booleana.

Sintaxis:

```
if B1 → S1
□ B2 → S2
...
□ Bn → Sn
fi
```

donde B1, B2, ..., Bn (llamadas **guardas**) son expresiones de tipo booleano, y S1, S2, ..., Sn son sentencias.

Semántica:

1. Se evalúan todas las guardas B1, ..., Bn usando el estado actual. (se obtienen valores booleanos).
2. Se elige alguna de las guardas que da True. Si ninguna da True, el programa **termina con error** ("se rompe"). Supongamos que elegimos la k-ésima guarda (Bk = True).
3. Ejecutar esa rama del if: Sk. (el estado final es el que resulta de esta ejecución).

Observación: **No determinismo:** Puede haber varias guardas que den True. En ese caso, la semántica dicta que se puede ejecutar cualquiera de esas (pero siempre una y sólo una). La semántica no me dice cuál va a ser (no necesariamente es la primera). El programador no elige.

Repetición, ciclo o bucle (do)

Sentencia compuesta. Me permite repetir la ejecución de una sentencia mientras se cumpla una condición.

Sintaxis:

do B $\rightarrow$ S od

donde B es una expresión booleana (**guarda**), y S es una sentencia (**cuerpo del ciclo**).

Semántica:

1. Se evalúa la guarda B en el estado actual.
2. Si da False, termina la ejecución.
Si da True, modifica el estado ejecutando S, y luego vuelve al punto 1.

Anotaciones de programa

Recordemos este programa:

```
Const N : Int ;
Var numero_actual, resultado : Int ;
l1 numero_actual, resultado := 0, 0 ;
l2 do numero_actual ≤ N →
l3   if numero_actual mod 6 = 0 →
l4     resultado := resultado + 1 ;
l5   □ numero_actual mod 6 ≠ 0 →
l6     skip
l7   fi ;
l8   numero_actual := numero_actual + 1
l9 od
```

línea	estado/guardas	observaciones
	$\sigma_0: N \mapsto 29,$ numero_actual $\mapsto -11,$ resultado $\mapsto 77$	estado inicial
l ₁ numero_actual, resultado := 0, 0 ;	$\sigma_1: N \mapsto 29,$ numero_actual $\mapsto 0,$ resultado $\mapsto 0$	inicialización
l ₂	$\sigma_1 // \text{numero_actual} \leq N \equiv$ True	
...		
l ₂	$\sigma_2: N \mapsto 29,$ numero_actual $\mapsto 1,$	

	resultado $\mapsto$ 1	
...		
ℓ_2	$\sigma_3: N \mapsto 29,$ numero_actual $\mapsto$ 2, resultado $\mapsto$ 1	
...		
ℓ_2	$\sigma_4: N \mapsto 29,$ numero_actual $\mapsto$ 3, resultado $\mapsto$ 1	
y así sucesivamente hasta terminar.		

En la línea 2, voy a tener una secuencia de estados cuyos valores para numero_actual serán 0, 1, 2, 3, ..., 29, 30.
N será siempre 29
resultado será: 1, 1, ..., 2, 2, ..., 3, ... 4, ... 5, ... , 5.

En general, en toda línea de mi programa voy a tener un conjunto de estados posibles en ese punto. Para describir propiedades que cumplen los estados posibles en un punto de mi programa puedo usar **predicados**.

Ejemplo: En la línea 2 puedo escribir los siguientes predicados:

- numero_actual $\geq$ 0
- $N = 29$ (sólo para este ejemplo, depende del estado inicial)
- numero_actual $\leq$ 30
- resultado $\geq$ 0
- resultado $\leq$ 30
- resultado $\leq 30 / 6$ (usando nociones de matemática)

Incluso podemos poner conjunciones de los anteriores:

- $0 \leq \text{numero_actual} \leq 30$
- $0 \leq \text{numero_actual} \leq 30 \wedge 0 \leq \text{resultado} \leq 30$

Incluso este predicado es correcto también:

- True

Incluso podemos usar expresiones cuantificadas para decir cosas más interesantes:

- resultado = $\langle N \mid i : 0 \leq i \leq N : i \bmod 6 = 0 \rangle$

¿es cierto esto en la línea 2? ¿vale en todo estado posible en la línea 2?

(segun este predicado, resultado debe valer 5 que sería en realidad el resultado final)

NO VALE SIEMPRE en la línea 2.

Lo corregimos un poco:

- resultado = $\langle N \mid i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle$

(estoy diciendo que en la línea 2 el estado es tal que en resultado he calculado un “**resultado parcial**”, que en este caso sería contar los múltiplos de 6 pero sólo hasta numero_actual-1).

Recordemos el enunciado: “Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.”

Reescribamos el programa pero con algunas anotaciones:

```
Const N : Int ;
Var numero_actual, resultado : Int ;
  { N > 0 }
// PRECONDICIÓN: esta anotación de programa me habla del estado inicial
l1 numero_actual, resultado := 0, 0 ;
  { numero_actual = 0 ∧ resultado = 0 }
l2 do numero_actual ≤ N →
  { resultado = ⟨ N i : 0 ≤ i < numero_actual : i mod 6 = 0 ⟩ }
l3   if numero_actual mod 6 = 0 →
l4     resultado := resultado + 1 ;
l5   [] numero_actual mod 6 ≠ 0 →
l6     skip
l7   fi ;
l8   numero_actual := numero_actual + 1
l9 od
  { numero_actual = N + 1 ∧ resultado = ⟨ N i : 0 ≤ i ≤ N : i mod 6 = 0 ⟩ }
// POSTCONDICIÓN: esta anotación de programa me habla del estado final
```

Una **anotación de programa** es un predicado que se puede introducir en un punto de un programa. Este predicado afirma algo que es satisfecho por todo estado posible en ese punto del programa.

Observaciones:

- Las anotaciones de programa **no son** parte de los programas.
- Las vamos a insertar en medio de los programas usando llaves “{”, “}”.
- Todo estado posible en ese punto del programa debe satisfacer el predicado, pero también el predicado puede admitir más estados que nunca suceden en ese punto.
- **Corolario:** El predicado “True” es siempre una anotación de programa correcta. (aunque es poco informativa)-
- **Corolario:** El predicado “False” nunca es una anotación de programa correcta.

Llamamos **precondición** a la anotación de programa que usamos para describir el estado inicial, o sea la que ubicamos en el punto inicial del programa.

Llamamos **postcondición** a la anotación de programa que usamos para describir el estado final, o sea la que ubicamos en el punto final del programa.

Observaciones:

- La precondition me dice cosas que puedo asumir acerca de la información de entrada de mi problema.
- La postcondition me sirve para expresar qué debe calcular el programa, o sea, qué problema está siendo resuelto.
- La precondition y las postcondition me sirven para **especificar**: Puedo traducir un enunciado informal en una especificación.

Especificación

Una **especificación** es una descripción formal del problema que se quiere resolver a través de una precondition y una postcondition.

Notación para especificar:

$\{ P \}$ ← precondition (predicado)
 S ← programa incógnita (letra S)
 $\{ Q \}$ ← postcondition (predicado)

A menudo incluiremos en la especificación las constantes y variables necesarias para especificar el problema.

Ejemplo: Para el problema ya visto: “Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.”

Una especificación posible es:

```

Const N : Int;
Var resultado : Int;
{ N > 0 }
S
{ resultado =  $\langle N \mid 0 \leq i \leq N : i \bmod 6 = 0 \rangle$  }
```

(observación: no se declara numero_actual, porque no es parte de la especificación si no de la solución.)

Referencias

[Libro: Cálculo de programas](#)

[Pizarra anterior](#)

[Pizarra siguiente](#)